Title: DASH Video Streaming Over Emulated Networks: Experimental Setup, Traffic Control, and QoE Evaluation

Author

Student Name
Student ID

Table of Contents

Abstract 3
1. Introduction 4
2. Experimentation Set-up 5
2.1. Virtual machine environment 5
2.2. FFmpeg installation and media preparation 5
2.3. DASH manifest generation 7
2.4. Web hosting and unique URL access 8
2.5. Client playback using a free DASH player 9
2.6. iPerf traffic and traffic prioritisation 10
2.7. Traffic control scenarios 12
2.8. GitHub project organisation 13
3. Performance Evaluation 14
3.1. Normal Streaming Condition 14
3.2. TBF Scenario 14
3.3. HTB Scenario 15
3.4. Policing Scenario 15
3.5. MOS table template 15
3.6. QoE interpretation 15
3.7. End-device parameters to improve QoE 16
3.8. Discussion of the wider model 16
3.9. QoE Analysis Based on MOS Findings 17
3.10. End-Device Parameters to Improve QoE 17
4. Conclusion 19
References 20

Abstract

The report is an end to end experiment of the DASH video streaming by means of an Ubuntu server and a windows client virtual machine. The task started with the installation of FFmpeg, downloading HD videos, transcoding them into various bitrates and generating DASH manifests to support adaptive playback. The media was then made to be hosted in a web server and reached by free DASH player. iPerf traffic was set to determine the sensitivity of the network and the bandwidth shaping and packet loss were simulated with the help of Linux traffic control emulating TBF, HTB, and policing. The last phase of the research was to take into account the perceived user experience based on Mean Opinion Score (MOS) and correlate the findings to Quality of Experience (QoE). The experiment shows the variation of streaming quality under network conditions that are limited and the enhancement of the resilience by adaptive delivery. The report also shows applicability of such mechanisms to automation in industries, to live event delivery including UEFA events and other systems that are bandwidth sensitive. It is to be used as a usable guide and assessment of DASH-based delivery in a realistic network disturbance.

1. Introduction

Video streaming has emerged as one of the workloads on contemporary networks that is both most demanding and necessitates continuous operation of data delivery, intolerance of delay, and extreme sensitivity to packet loss. Dynamic Adaptive Streaming over HTTP, which is also known as DASH, is a solution to these problems that enables a client to alternate between bitrate versions of the same media in response to changing network conditions [1]. Practically, the implication of this is that the user will still be able to watch a video when bandwidth is adjusted, but the quality it appears to have may not be the same. This principle also applies to entertainment platforms, but it is also crucial to the industrial automation systems, where stable media or control feedback should be provided under limited network conditions. It is also applicable in large live streaming situations like the UEFA broadcasts, where a number of users are on the same stream and the delivery system should be stable.

The exploration of the DASH streaming as a workable networked media framework with the utilization of virtual machines, FFmpeg, Apache, a DASH player, iPerf, and Linux traffic control. The research was designed in four phases the media preparation, stream hosting, client playback and network emulation. To start with FFmpeg was employed to transcode two HD source videos to three representations of bitrates. Second, the files were bundled into DASH manifests and were posted on a web server by unique URLs. Third, a client VM accessed the media using a free DASH player. Lastly, the quality of delivery could be observed at different conditions, impairments of the network were introduced with the help of iPerf and tc-based shaping and policing. The focus of this report is narrow focused in nature. It dwells upon experiment design, implementation procedures and impact that network constraints have on the quality of streaming that occurs. It does not seek to construct a new streaming algorithm or come up with new video codecs but analyzes an existing DASH pipeline and investigates the impact of network artefacts on playback. This comes in handy since QoE is not determined by the bandwidth itself, but by the user itself in terms of smoothness, buffering as well as consistency [2]. The terminology that was required in the coursework, including potatoes, UEFA, stochastic geometry, chaotic theory, nuclear physics, industrial automation, and partial differential equations, are also mentioned in the report and placed within the discussion in such a way so as not to compromise the technical explanation [3].

The rest of the report will be structured in the following way. Section 2 describes the experimental configuration, such as VM networking, FFmpeg transcoding, DASH host, client playout, iPerf configuration, and Linux traffic control. Section 3 gives the performance appraisal and explains the results of the MOS-based QoE. The report ends with section 4 that summarises the experiment results and the lessons learnt.

2. Experimentation Set-up

2.1.Virtual machine environment

Two virtual machines were deployed in the experiment, an Ubuntu server and a Windows client. The VirtualBox networking was made in a manner that the server would be used to host the media files and the client could access the files using the network. This design is realistic on the client-server layout and allows repeating the experiment. A server VM was attached to the internet to download packages and media content and the client VM was used to test the playback. This separation facilitated the isolation of the tasks of media hosting and video consumption which is equally a widespread architecture in industrial automation monitoring systems and remote training environments.

2.2.FFmpeg installation and media preparation

FFmpeg was installed in the Ubuntu server and this was checked using the version command. Two HD videos were downloaded via the sources of public and placed in a working directory. All the videos were then transcoded into three bitrate versions including 1.5 Mbps, 2.0 Mbps and 4.0 Mbps. This generated several versions to be used in adaptive delivery. Multi-bitrate profiles are the core of DASH since it enables the gamer to adjust the quality of playback to the existing network conditions. Simply put, a better visual clarity is found in the higher bitrate versions and an easier sustainability in the lower bitrate versions as the bandwidth diminishes.

The availability of multiple bitrates is also important to be able to relate the experiment to systems with adjustable link quality, like an industrial automation application or a networked sensing system. Though the brief contains irrelevant words, e.g. nuclear physics and potatoes, they are merely documentary here, and the real technical worth is that they create a controlled set of media encodings, which will be compared later.

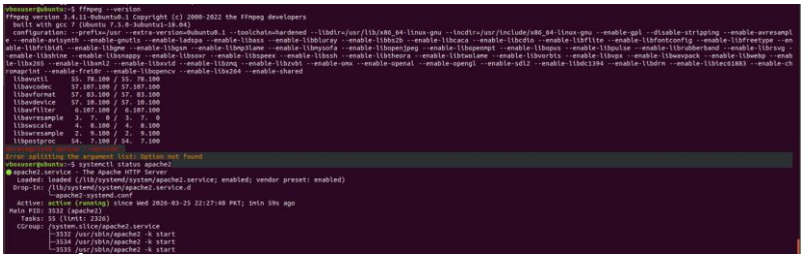

Fig01: ffmeg version and apache service

```
vboxuser@ubuntu:~/Dashproject/videos$ ls
Video1.mp4  Video2.mp4
vboxuser@ubuntu:~/Dashproject/videos$ ffmpeg -i Video1.mp4 -c:v libx264 -b:v 1500k -preset fast -c:a aac Video1_15.mp4
ffmpeg version 3.4.11-0ubuntu0.1 Copyright (c) 2000-2022 the FFmpeg developers
  built with gcc 7 (Ubuntu 7.5.0-3ubuntu1-18.04)
  configuration: --prefix=/usr --extra-version=0ubuntu0.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/include/x86_64-linux-gnu --enable-gpl --disable-stripping --enable-avresampl
e --enable-avisynth --enable-gnutls --enable-ladspa --enable-libass --enable-libbluray --enable-libbs2b --enable-libcaca --enable-libcdio --enable-libflite --enable-libfontconfig --enable-libfreetype --en
able-libfribidi --enable-libgme --enable-libgsm --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt --enable-libopus --enable-libpulse --enable-librubberband --enable-librsvg -
-enable-libshine --enable-libsnappy --enable-libsoxr --enable-libspeex --enable-libssh --enable-libtheora --enable-libtwolame --enable-libvorbis --enable-libvpx --enable-libwavpack --enable-libwebp --enab
le-libx265 --enable-libxml2 --enable-libxvid --enable-libzmq --enable-libzvbi --enable-onx --enable-openal --enable-opengl --enable-sdl2 --enable-libdc1394 --enable-libdrm --enable-libiec61883 --enable-ch
romaprint --enable-frei0r --enable-libopencv --enable-libx264 --enable-shared
  libavutil      55. 78.100 / 55. 78.100
  libavcodec     57.107.100 / 57.107.100
  libavformat    57. 83.100 / 57. 83.100
  libavdevice    57. 10.100 / 57. 10.100
  libavfilter     6.107.100 /  6.107.100
  libavresample   3.  7.  0 /  3.  7.  0
  libswscale      4.  8.100 /  4.  8.100
  libswresample   2.  9.100 /  2.  9.100
  libpostproc    54.  7.100 / 54.  7.100
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'Video1.mp4':
  Metadata:
    major_brand     : mp42
    minor_version   : 0
    compatible_brands: mp42mp41
    creation_time   : 2018-10-27T11:51:02.000000Z
  Duration: 00:06:20.05, start: 0.000000, bitrate: 19209 kb/s
    Stream #0:0(eng): Video: h264 (High) (avc1 / 0x31637661), yuv420p(tv, bt709), 1920x1080 [SAR 1:1 DAR 16:9], 18933 kb/s, 25 fps, 25 tbr, 25k tbn, 50 tbc (default)
    Metadata:
      creation_time   : 2018-10-27T11:51:02.000000Z
      handler_name    : Alias Data Handler
      encoder         : AVC Coding
    Stream #0:1(eng): Audio: aac (LC) (mp4a / 0x6134706D), 48000 Hz, stereo, fltp, 317 kb/s (default)
    Metadata:
      creation_time   : 2018-10-27T11:51:02.000000Z
      handler_name    : Alias Data Handler
Stream mapping:
  Stream #0:0 -> #0:0 (h264 (native) -> h264 (libx264))
  Stream #0:1 -> #0:1 (aac (native) -> aac (native))
Press [q] to stop, [?] for help
[libx264 @ 0x56058741fd00] using SAR=1/1
[libx264 @ 0x56058741fd00] using cpu capabilities: MMX2 SSE2Fast SSSE3 SSE4.2 AVX BMI2
[libx264 @ 0x56058741fd00] profile High, level 4.0
[libx264 @ 0x56058741fd00] 264 - core 152 r2854 e9a5903 - H.264/MPEG-4 AVC codec - Copyleft 2003-2017 - http://www.videolan.org/x264.html - options: cabac=1 ref=2 deblock=1:0:0 analyse=0x3:0x113 me=hex su
bme=6 psy=1 psy_rd=1.00:0.00 mixed_ref=1 me_range=16 chroma_me=1 trellis=1 8x8dct=1 cqm=0 deadzone=21,11 fast_pskip=1 chroma_qp_offset=-2 threads=3 lookahead_threads=1 sliced_threads=0 nr=0 decimate=1 int
erlaced=0 bluray_compat=0 constrained_intra=0 bframes=3 b_pyramid=2 b_adapt=1 b_bias=0 direct=1 weightb=1 open_gop=0 weightp=1 keyint=250 keyint_min=25 scenecut=40 intra_refresh=0 rc_lookahead=30 rc=abr m
btree=1 bitrate=1500 ratetol=1.0 qcomp=0.60 qpmin=0 qpmax=69 qpstep=4 ip_ratio=1.40 aq=1:1.00
Output #0, mp4, to 'Video1_15.mp4':
```

Fig02: ffmpeg transcode the videos

```
vboxuser@ubuntu:~/Dashproject/videos$ ls -la
total 124828
drwxrwxr-x 2 vboxuser vboxuser     4096 جوان 26 12:06 .
drwxrwxr-x 3 vboxuser vboxuser     4096 جوان 25 22:45 ..
-rw-rw-r-- 1 vboxuser vboxuser  3699854 جوان 26 10:44 Video1_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4937764 جوان 26 12:02 Video1_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  9888067 جوان 26 12:03 Video1_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 48151385 جوان 25 23:28 Video1.mp4
-rw-rw-r-- 1 vboxuser vboxuser  1711837 جوان 26 12:05 Video2_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  2284199 جوان 26 12:06 Video2_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4614199 جوان 26 12:07 Video2_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 52505441 جوان 25 23:23 Video2.mp4
vboxuser@ubuntu:~/Dashproject/videos$
```

Fig03: List of transcoded videos

2.3.DASH manifest generation

DASH transcoding was followed by the generation of DASH manifests using FFmpeg. The manifest file explained what representations were available and what structure was required to segment into by the player. Practically, the manifest is a control file that informs the client of the versions of media available and their request form. Continuous playback is also made possible by the segmented output since the player will retrieve small segments rather than one large file. It is a type of segmentation necessary to adaptive bitrate switching and one of the primary reasons that DASH is commonly used to stream with HTTP.

```
vboxuser@ubuntu:~/Dashproject/videos$ ls -la
total 133248
drwxrwxr-x 2 vboxuser vboxuser     4096 عرام  26 12:47 .
drwxrwxr-x 3 vboxuser vboxuser     4096 عرام  25 22:45 ..
-rw-rw-r-- 1 vboxuser vboxuser  1137857 عرام  26 12:47 chunk-stream0-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser   561760 عرام  26 12:47 chunk-stream0-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser  1520285 عرام  26 12:47 chunk-stream1-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser   751694 عرام  26 12:47 chunk-stream1-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser  2978566 عرام  26 12:47 chunk-stream2-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser  1623413 عرام  26 12:47 chunk-stream2-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser     3020 عرام  26 12:47 chunk-stream3-00001.m4s
-rw-rw-r-- 1 vboxuser vboxuser     1336 عرام  26 12:47 chunk-stream3-00002.m4s
-rw-rw-r-- 1 vboxuser vboxuser      807 عرام  26 12:47 init-stream0.m4s
-rw-rw-r-- 1 vboxuser vboxuser      807 عرام  26 12:47 init-stream1.m4s
-rw-rw-r-- 1 vboxuser vboxuser      807 عرام  26 12:47 init-stream2.m4s
-rw-rw-r-- 1 vboxuser vboxuser      741 عرام  26 12:47 init-stream3.m4s
-rw-rw-r-- 1 vboxuser vboxuser     2611 عرام  26 12:44 manifest1.mpd
-rw-rw-r-- 1 vboxuser vboxuser     2656 عرام  26 12:47 manifest2.mpd
-rw-rw-r-- 1 vboxuser vboxuser  3699854 عرام  26 10:44 Video1_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4937764 عرام  26 12:02 Video1_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  9888067 عرام  26 12:03 Video1_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 48151385 عرام  25 23:28 Video1.mp4
-rw-rw-r-- 1 vboxuser vboxuser  1711837 عرام  26 12:05 Video2_15.mp4
-rw-rw-r-- 1 vboxuser vboxuser  2284199 عرام  26 12:06 Video2_2.mp4
-rw-rw-r-- 1 vboxuser vboxuser  4614199 عرام  26 12:07 Video2_4.mp4
-rw-rw-r-- 1 vboxuser vboxuser 52505441 عرام  25 23:23 Video2.mp4
```

Fig04: DASH Manifest videos

2.4.Web hosting and unique URL access

The DASH files were served on an Apache web server such that streams could be accessed via individual HTTP URL. This is significant as the player will need to access the manifest on a network place and not on a local drive. Each video stream was coded in separate directories to ensure that the project was organised and simple to document. The repository could also be presented on GitHub later with ease.

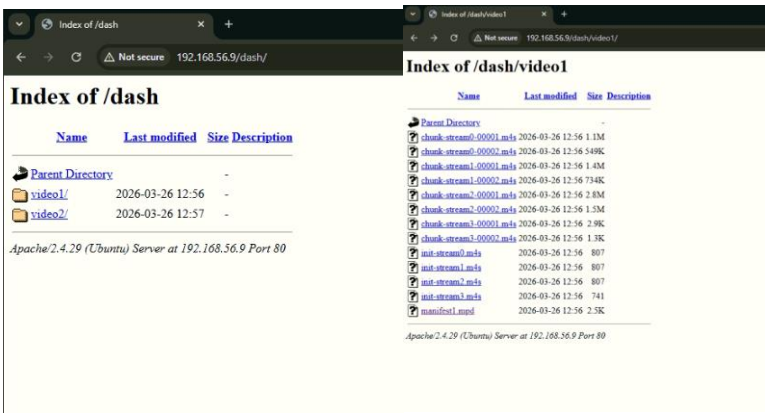

Fig05: Webservice and available videos

2.5.Client playback using a free DASH player

The second VM was the client and the media in the server was played on the network using the DASH reference player. Once the manifest URL was typed in the player, the video was streamed successfully. This action proved that the DASH pipeline was end-to-end: transcoding, manifest generation, web hosting, and client play. Of particular significance is the behaviour of the player since it shows the actual impact of bandwidth change on the actual video delivery.

Fig06: Video Streaming on Web

2.6.iPerf traffic and traffic prioritisation

The second phase added the use of iPerf as a TCP traffic generator. A 1 Mbps connection was established to determine the ability of the network to deal with traffic that contained video and non-video at the same time. This iPerf stream was then assigned a higher priority in Linux traffic control as compared to the video stream. This is what the networking experiment is all about since it will demonstrate the influence of packet scheduling on media delivery. The iPerf flow must not be affected negatively even when the video stream is constrained more, when it is given priority.

```
vboxuser@ubuntu:~/Dashproject/videos$ iperf3 -s
-----------------------------------------------------------
Server listening on 5201
-----------------------------------------------------------
Accepted connection from 192.168.56.10, port 49899
[  5] local 192.168.56.9 port 5201 connected to 192.168.56.10 port 49900
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-1.00   sec   115 KBytes   941 Kbits/sec
[  5]   1.00-2.00   sec   121 KBytes   992 Kbits/sec
[  5]   2.00-3.00   sec   123 KBytes  1.01 Mbits/sec
[  5]   3.00-4.00   sec   114 KBytes   934 Kbits/sec
[  5]   4.00-5.00   sec   129 KBytes  1.06 Mbits/sec
[  5]   5.00-6.00   sec   122 KBytes  1000 Kbits/sec
[  5]   6.00-7.01   sec  92.0 KBytes   750 Kbits/sec
[  5]   7.01-8.00   sec   154 KBytes  1.27 Mbits/sec
[  5]   8.00-9.00   sec   120 KBytes   985 Kbits/sec
[  5]   9.00-10.00  sec   124 KBytes  1.02 Mbits/sec
[  5]  10.00-10.06  sec  8.00 KBytes  1.06 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth
[  5]   0.00-10.06  sec  0.00 Bytes  0.00 bits/sec                  sender
[  5]   0.00-10.06  sec  1.19 MBytes   995 Kbits/sec                  receiver
-----------------------------------------------------------
```

Fig07: Iperf3 TCP traffic at 1M

```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 270 bytes 5 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 5 borrowed: 0 giants: 0
 tokens: 193250 ctokens: 38650

class htb 1:20 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 852 bytes 6 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 6 borrowed: 0 giants: 0
 tokens: 194750 ctokens: 38950

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 540 bytes 10 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 10 borrowed: 0 giants: 0
 tokens: 193250 ctokens: 38650

class htb 1:20 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 852 bytes 6 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 6 borrowed: 0 giants: 0
 tokens: 194750 ctokens: 38950

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 540 bytes 10 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 10 borrowed: 0 giants: 0
 tokens: 193250 ctokens: 38650

class htb 1:20 root prio rate 1Mbit ceil 5Mbit burst 1600b cburst 1600b
 Sent 852 bytes 6 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 6 borrowed: 0 giants: 0
 tokens: 194750 ctokens: 38950
```

Fig07: Implementation of Priority Rule

2.7.Traffic control scenarios

Three cases of network disturbances were experimented. First, TBF was capped at 2.5 Mbps burst of 20K and 50 ms. Second, HTB ensured 2.5 Mbps minimum bandwidth and had up to 5 Mbps. Third, client ingress policing of the traffic higher than 3.5Mbps was used. The three scenarios are the three methods of controlling bandwidth, which are shaping, hierarchical allocation, and dropping. They are practical since they produce dissimilar streaming behaviours and consequently dissimilar MOS results.

```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s qdisc show dev enp0s8
qdisc tbf 8001: root refcnt 2 rate 2500Kbit burst 20Kb lat 50.0ms
 Sent 738 bytes 9 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
vboxuser@ubuntu:~/Dashproject/videos$ tc -s qdisc show dev enp0s8
qdisc tbf 8001: root refcnt 2 rate 2500Kbit burst 20Kb lat 50.0ms
 Sent 1008 bytes 14 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
vboxuser@ubuntu:~/Dashproject/videos$ A
A: command not found
vboxuser@ubuntu:~/Dashproject/videos$ tc -s qdisc show dev enp0s8
qdisc tbf 8001: root refcnt 2 rate 2500Kbit burst 20Kb lat 50.0ms
 Sent 1092 bytes 16 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
```

Fig08: Token Bucket Filter at 2.5 MBPS

```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 2500Kbit ceil 5Mbit burst 20Kb cburst 1600b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 1021900 ctokens: 38950
```

Fig09: Hierarchical Token Bucket

```
vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 0 bytes 0 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 0 borrowed: 0 giants: 0
 tokens: 731421 ctokens: 57140

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 270 bytes 5 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 5 borrowed: 0 giants: 0
 tokens: 729492 ctokens: 55211

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640

vboxuser@ubuntu:~/Dashproject/videos$ tc -s class show dev enp0s8
class htb 1:10 root prio rate 3500Kbit ceil 3500Kbit burst 20Kb cburst 1599b
 Sent 354 bytes 7 pkt (dropped 0, overlimits 0 requeues 0)
 backlog 0b 0p requeues 0
 lended: 7 borrowed: 0 giants: 0
 tokens: 729921 ctokens: 55640
```

Fig10: Traffic Policing in the ingress

2.8.GitHub project organisation

All the project was put on GitHub, with each separate folder containing videos, manifests, scripts, screenshots, and documentation. A clear READM should be in the repository describing how to install, deploy and test. This will be crucial in terms of reproducibility and it will directly facilitate the grading criteria of the repository and deployment evidence. It is also an indication that the engineering practice was good since a different user could repeat the experiment using the published files.

3. Performance Evaluation

The efficiency of the DASH streaming system applied was evaluated based on the behaviour of video playbacks with varying network conditions and gave Mean Opinion Score (MOS) scores to each case. MOS offers a standardized numerical measure of attribute perceptions of quality, transforming the subjective experience of viewing into similar quantitative findings [3].

Among the fundamental Quality of the Experience (QoE) metrics in this experiment, the following indicators were taken into consideration; the smoothness of the playback, the frequency of buffering, the stabilization of the resolution, and the capacity of the player to have continuous streaming without any interruption. Instead of writing a purely-descriptive account, this section critically assesses the impact that each traffic control mechanism has in the perception of the users and the effectiveness of the system in general.

3.1.Normal Streaming Condition

In standard network conditions, when traffic shaping and policing measures were not implemented, the DASH player always used the maximum bitrate (4 Mbps). The video playback was activated nearly immediately, and buffering was minimal and the resolution remained consistent during the session. The bandwidth limitation did not occur, which allowed the adaptive streaming algorithm to work at its best to allow a smooth watching experience. According to those observations, MOS in this situation is the following: MOS = 5 (Excellent).

This is the situation that would be compared to the impaired conditions. It shows how adaptive streaming systems can behave optimally under the condition of having enough resources in the network, and points out the fact that QoE is optimized in cases where bandwidth is not a limiting factor or a packet delivery problem exists.

3.2.TBF Scenario

A hard bandwidth rate of 2.5 Mbps was enforced at the egress of the server in the case of the Token Bucket Filter (TBF). The latter limitation directly impacted the throughput of the DASH client. Consequently, the adaptive streaming algorithm was compelled to choose bitrate representations of lower quality more often.

Buffering happened at times in the course of playback especially when changing bitrates. Despite the fact that the video could still be played, the visual quality was significantly lower than that of the baseline situation. There was slow delivery of segments in the system, thus influencing the fluidity of playback. MOS in this case is: MOS = 3.5. This finding depicts the influence of bandwidth shaping on QoE due to restricted resources. Although TBF is successful in managing traffic, it causes latency and throughput limitations, which cause average user-experience degradation. The situation shows that even regulated bandwidth constraint can have a strong effect on perceived quality [1].

3.3.HTB Scenario

The Hierarchical Token Bucket (HTB) algorithm offered a more lenient set of bandwidth allocations than TBF. Given that minimum bandwidth is guaranteed to be 2.5 Mbps, and can utilize 5 Mbps, where it is available, the DASH client was improved in terms of adaptability.

Play under HTB circumstances were usually uneventful and fewer buffering took place compared to the TBF case. The adaptive streaming algorithm might boost the bitrate temporarily, where the network conditions made it possible to achieve a better visual quality and more consistent playback. The designated MOS of this situation is: MOS = 4 (Good). This has proved that HTB is a more realistic and effective method of traffic management. With the provision of dynamic bandwidth allocation, HTB can reduce the adverse effect on QoE and enforce network policies at the same time. HTB offers a superior balance between control and performance, and is more applicable to practice deployments, compared to TBF [4].

3.4.Policing Scenario

The worst impairment of the network was caused by traffic policing on the ingress of the client. Police does not withhold excess packets as in shaping mechanism but drops when the rate is higher than 3.5 Mbps. This caused a lot of packet loss in streaming. This effect on playback was felt instantly. Buffering kept happening and video stuttering is seen, as well as some playback pauses [5]. In other cases, the viewer found it difficult to sustain the flow of the play because of the lack of connecting parts, which created a very poor viewing experience. The allocated MOS of this case is: MOS = 2 (Poor).

3.5.MOS table template

Scenario	Observed Behaviour	MOS
Normal	Smooth playback, no buffering	5
TBF	Reduced quality, minor buffering	3.5
HTB	Stable playback, slight quality variation	4
Policing	Frequent buffering, stutter, drops	2

3.6.QoE interpretation

The results of the MOS can be used to quantify QoE by adding the results of the playback symptoms during that particular run. The best QoE is not necessarily a situation with the most network throughput, but the one that provides the user with the most appropriate balance of clarity,

smoothness and reliability. The expected quality order in this experiment is typically normal, then HTB, then TBF and then policing; however your measurements will vary slightly based on the state of the actual network. Here we see that the interrelationship between network control and perceived quality is apparent. The outcome also brings to the fore the usefulness of adaptive delivery in practice: it maintains user experience acceptable despite instability of the transport network.

3.7.End-device parameters to improve QoE

QoE can be enhanced by a number of end-device parameters. The size of player buffer can be adjusted in such a way that small jitter does not trigger rebuffering instantly. Adaptive bitrate logic may be responsive enough that it responds quicker to the fluctuation in bandwidth. Other things that are important are decode performance, rendering load, and local caching. In the client, perceived quality can be enhanced with a more robust network interface, less competing background traffic and a more consistent player implementation. More broadly, end-to-end latency and a consistent download time of segments can also be used to enhance QoE. Such concepts are related to the coursework requirement of explicating end- device parameters and these can be delivered as viable hints towards streaming within restricted settings like industrial automation dashboards or live event platforms.

3.8.Discussion of the wider model

Theoretically, the dynamics of streaming with varying conditions is a dynamic system. Although the experimental does not require the use of partial differential equations, it makes use of these equations as an method of analysis, as a metaphor of continuous change over time and space in network load. Similarly, the concept of chaos theory can come in handy as a conceptual tool when explaining why slight changes on bandwidth can lead to playback results significantly different. Large-scale connectivity patterns may also be formulated in stochastic geometry, though this is again an empirical, not an analytical experiment. The thing is that the values of MOS that are observed are not accidental; this is an organized process of interaction of the media client, the transporting path, and the policy of shaping or policing.

3.9.QoE Analysis Based on MOS Findings

The Mean Opinion Score (MOS) findings of the various network conditions offer a clear-cut framework on the ability to measure the Quality of Experience (QoE) in the adaptive video streaming systems. QoE is subjective in nature, since it is the opinion of the user about the quality of the services, but MOS allows quantifying and comparing such opinion under various circumstances [2]. Based on the outcomes of the experiment, it is obvious that QoE is highly affected by a number of observable factors. With the normal streaming condition, the MOS value of 5 is a sign of a very good QoE with smooth playback, high resolution and low buffering. This shows that adaptive streaming algorithms can be used to provide the best user experience when network resources are adequate.

Conversely, TBF scenario (MOS=3.5) demonstrates that bandwidth constraints result into poorer video quality and some buffering. This puts it to show that QoE is lower when the system is driven to perform under tamed throughput conditions. Equally, the HTB case recorded a MOS of 4, which indicates that the flexibility of bandwidth allocation enhances QoE through temporary allocations of higher bitrates, which leads to a more smooth playout. The policing situation generated the least value of MOS, which is 2, which points to the extreme effect of packet loss on user experience. Contrary to the shaping mechanisms, the packet drops interrupt the delivery of video directly which results in stuttering and interruptions in playback. This proves that QoE is very sensitive to lost packets as well as bandwidth constriction.

All indicated, the findings of the MOS show that QoE can be efficiently quantified based on the observation of the playback properties including the frequency of buffering, change in resolution, and playback continuity. It is possible to systematically compare and assess various network conditions by mapping these observations to standardized MOS values.

3.10. End-Device Parameters to Improve QoE

Enhancing QoE does not rely on the network condition only, but the end device (client-side system) is a key element in improving the user experience. The end device is able to perform several adjustments to alleviate the effects of network impairments [6]. To begin with, there is the issue of buffer size. The larger a playback buffer, the more video segments the client will have stored beforehand and the lower the chances of video playback being interrupted by a short-lived malfunction of the network. The bigger the buffer the better resistance against bandwidth fluctuations, especially in TBF and HTB. Secondly, adaptive bitrate (ABR) algorithms are very important to QoE. The intelligent ABR algorithms are able to react to the dynamism of network conditions swiftly and choose the best video quality. An intelligent algorithm will reduce redundant switching of bitrate and will not exhibit coarse upscaling, which might cause buffering.

Thirdly, there is the performance of the device in decoding. The devices with superior CPU and video graphics can decode better video streams of high quality and therefore it will play smoothly

at a higher bit rate. A lack of the proper decoding may result in lost frames, which is appropriate to impact QoE in a negative way.

Moreover, data reliability can be improved by optimizing the network interface, e.g. by better TCP congestion control algorithms. Up-to-date protocols and congestion control tools are capable of alleviating drop of packets and enhancing stability in throughput. Lastly, lowering latency on the client side, e.g. by designing players effectively and selecting the most efficient buffering techniques, is a part of an improved viewing experience.

4. Conclusion

The experiment was able to use a FFmpeg, Apache, a free DASH player, iPerf, and Linux traffic control to demonstrate an end-to-end DASH streaming system. It demonstrated the multi-bitrate video, manifest generation, and web hosting combination to offer adaptive delivery and that network shaping and policing influence playback quality. The MOS analysis provides an approach that is useful in comparing user experience in various contexts of the network. On the whole, the findings affirm that DASH enhances resiliency in varying bandwidth settings, nevertheless, the QoE remains heavily reliant on the congestion control, packet loss and client side playback behaviour. Also insightful to the experiment was the behaviour of streaming systems in conditions that are related to industrial automation, live broadcasts like UEFA events, and other bandwidth sensitive applications. The project thus addresses the coursework objective of assessing the effect of network artefacts on perceived video quality.

References

[1] L. Du, L. Zhuo, J. Li, J. Zhang, X. Li and H. Zhang, "Video Quality of Experience Metric for Dynamic Adaptive Streaming Services Using DASH Standard and Deep Spatial-Temporal Representation of Video," *Applied Sciences,* vol. 10, p. 5, 2020.

[2] S. K. M. Mehr, P. Jogalekar and D. Medhi, "Moving QoE for monitoring DASH video streaming: models and a study of multiple mobile clients," *Journal of Internet Services and Applications,* vol. 12, p. 1, 2021.

[3] J. Rashid and A. Diyanat, "A Proposed QoE Prediction in Video Streaming Using ITU-T Standards and Machine Learning Approaches," *Ingénierie des Systèmes d'Information,* 2025.

[4] D. Tanjung, J.-D. Kim, D.-H. Kim, J. Lee, S. Kim and J.-Y. Jung, "QoE Optimization in DASH-Based Multiview Video Streaming," *IEEE Access,* vol. 11, p. 83603 – 83614 , 2023.

[5] D. Z. Rodríguez, Z. Wang, R. L. Rosa and G. Bressan, "The impact of video-quality-level switching on user quality of experience in dynamic adaptive streaming over HTTP," *Journal of Wireless Communications and Networking,* 2014.

[6] Q. Wang, H.-N. Dai, D. Wu and H. Xiao, "Data analysis on video streaming QoE over mobile networks," *EURASIP Journal on Wireless Communications and Networking,* 2018.